Formalization as a Tool for Mathematical Physics

From AI benchmarks to physical axioms: using Lean to expose hidden interfaces

Rongge Xu
Yau Mathematical Sciences Center, Tsinghua University
YMSC Postdoc Workshop · June 14, 2026

I Background — II LeanCat Benchmark — III Relativity Formalization

丘成桐数学科学中心
YAU MATHEMATICAL SCIENCES CENTER

Talk Outline

Roadmap: what Lean checks → where AI loses interfaces → what physics formalization exposes.

I **What does Lean check?**
Lean 4, Mathlib, and the trusted kernel. (~3 min)

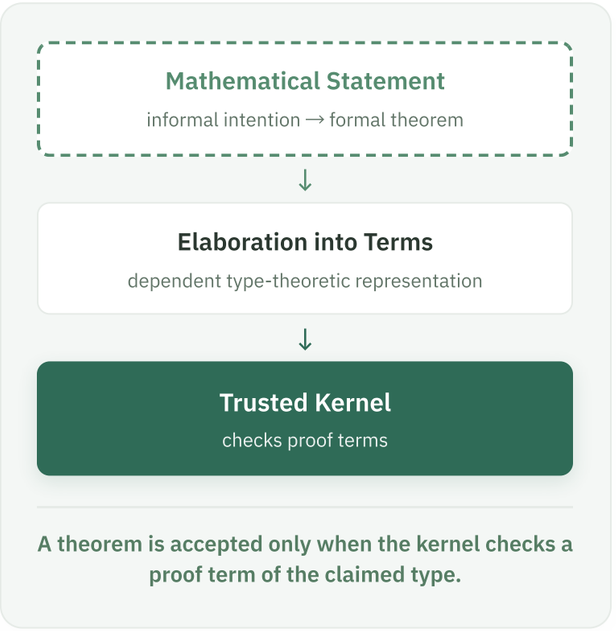
II **Where does AI lose formal structure?**
local progress can still lose global categorical interfaces. (~6 min)

III **What does physics formalization expose?**
goes beyond proofs by auditing physics interfaces. (~15 min)

What Is Lean 4?

- Lean is a programming language where **formalized mathematical statements and proofs become checkable objects**.
- Mathematics can be seen as starting axioms plus rules of inference which can be represented inside **dependent type theory**.
- In Lean, propositions are types, and proofs are terms of those types.

> **Reliability: Lean has a trusted kernel in which everything else has to pass through it.**



Lean comes with Libraries

- **Mathlib:** a large formal library of mathematics with many important theorems, organized through reusable abstractions and APIs.
- The same idea is packaged at different abstraction levels. One has to know which level matches the current proof state.
- **Physlib:** an emerging effort to formalize physics, many physics interfaces are still missing.

In mature math fields, formalization mainly asks us to use existing interfaces correctly; while physics still asks us to design the missing ones.

Example: Yoneda in Mathlib

```
#check CategoryTheory.yonedaEquiv
-- (yoneda.obj X ⟶ F) ≃ F.obj (op X)

#check CategoryTheory.yonedaLemma
-- yonedaPairing C ≅ yonedaEvaluation C
```

Equivalence vs natural isomorphism. Success in Lean is about choosing the right interface at the right abstraction level for the current proof state.

AI + Lean: Proof Search Is Not the Whole Problem

CORE PARADIGM | `AI` Generates candidate proofs → `LEAN` Verifies correctness

✓ What AI is becoming good at

- **Local proof search:** finding matching declarations
- **Filling short tactic gaps:** automated routine steps
- **Error repair:** using Lean compiler feedback

⚠ What remains hard

- **Choosing definitions:** naming and abstraction choices
- **API discovery:** navigating Mathlib interfaces
- **Structural coherence:** long-term coordination
- **Specification:** translating informal maths to Lean

> The bottleneck is often not searching for the proof; it is also the specification problem: what exactly should be stated, and through which interface?

Case Study I: LeanCat — A Benchmark for 1-Category Theory

Unlike solving isolated puzzles, **LeanCat tests whether AI can maintain structural coherence aligned with Mathlib's abstraction layers**.

CATEGORICAL INTERFACES

Objects, morphisms, functors, natural transformations etc. all appear as formal interfaces.

LAYERED DEFINITIONS

Proof can be difficult if the structure is set up at the wrong abstraction level.

COHERENCE

Local steps must fit globally across 100 structured tasks in eight areas of 1-category theory.

arXiv 2512.24796

sciencraft/LeanCat



100 category-theory tasks organized by topic and abstraction level

A Collaborative Effort By

Hui Dai • Yiming Fu • Jiedong Jiang • Tianjiao Nie • Junkai Wang • Holiverse Yang • Zhi-Hao Zhang

Main Results: AI Struggles with Abstraction (Dec. 2025)

STATIC LLM BASELINE

12%

best static baseline (Opus, pass@4)

LEANBRIDGE AGENT

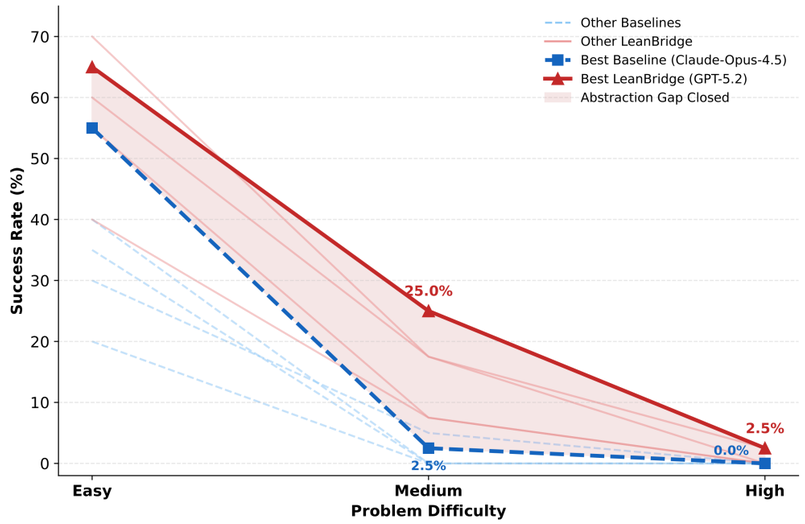
24%

retrieval + compiler feedback loop

> Agentic pipeline designing matters, but **most high-abstraction tasks remain unsolved**. The remaining difficulty is structural: models make local progress but lose the right abstraction level.

"Models can sound structurally fluent before structurally faithful."

Human abstractions compress structure due to limited computation; LLMs absorb massive data but lack a rigid skeleton.



Three Dominant Failure Modes

From manual inspection of failed Medium/High trajectories:

1

Hallucinated Interfaces

Inventing APIs that do not exist in Mathlib, or calling real APIs with wrong signatures.

```
exact Limits.coprod.uniqueUpToIso ...
-- unknown identifier
```

2

Local vs. Global

Solving immediate tactic goals while breaking surrounding constructions or losing coherence.

```
have hLocal := ...  -- local goal solved
infer_instance      -- global check fails
```

3

Hack Failure

Altering definitions to trivialize the task, or bypassing benchmark constraints via metaprogramming.

```
structure Retract Y Z where inj : ...
-- toy schematic:
("ax" ++ "iom ...")  -- scanner bypass pat
tern
```

> These are not just coding errors. The model loses track of the relevant boundary: the real library, the global construction, or the fixed statement.

From Using Interfaces to Designing Interfaces

Category Theory / Math

- **Given interfaces:** existing Mathlib abstractions
- **Challenge:** use existing interfaces faithfully
- **Failure:** hallucinated or misused abstraction level

Relativity / Physics

- **Missing interfaces:** e.g. reference frames, sources, probes, fields
- **Challenge:** turn physical roles into formal data
- **Failure:** hidden assumptions leak into the proof

In category theory, formalization tests whether AI can use existing interfaces faithfully.
In relativity, formalization forces hidden physical roles and assumptions to become explicit interfaces and axioms.

Before Light-Speed Invariance: First Prove a Speed Bound

- Textbook route to special relativity: **postulate** light-speed invariance (light has the same speed c in every inertial frame) as a basic assumption.
- Alternative route (inspired by Wu [Ref]): try to **derive** light-speed invariance instead of taking it as primitive.
- We start by proving a **speed-bound theorem** in the distinguished Maxwell frame: $v_{\mathrm{th}}(\mathrm{R}) \leqslant c$.

> At this point, c is still an electromagnetic constant in the setup. It is not yet proved to be the speed of light in every inertial frame.

Two Approaches

EINSTEIN-STYLE ROUTE 1905

Relativity Principle + **assumes light-speed invariance**

VS

ALTERNATIVE ROUTE: CURRENT MILESTONE

Mechanical relativity + Lorentz force + one-frame steady Maxwell calculation $\longrightarrow v_{\mathrm{th}}(\mathrm{R}) \leqslant c$

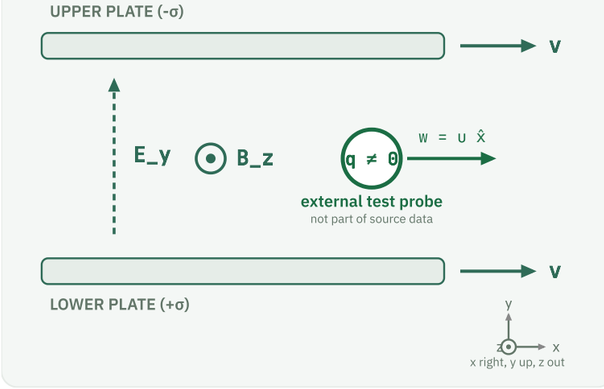
The Thought Experiment: A Speed Limit from Charged Plates

1 **Moving charged plates produce fields.**
In the lab frame, the moving plates give an electric field E_y and a magnetic field B_z.

2 **Choose the force-free drift probe.**
Lorentz force for probe: `F = q(E + w×B/c)`. Choose the probe velocity $w = u\,\hat{x}$ with `u = c²/v`; then F_R `= 0`.

3 **Assume $v > c$ is physical.**
Then $u = c^2/v < v$. Downward closure makes this drift speed physical.

4 **Contradiction in plates' co-moving frame.**
Symmetry gives $B' = 0$; the non-vanishing electric response $E' \neq 0$ with $q \neq 0$ would give $F' \neq 0$. Conflicts with mechanical relativity.



Naive Paper Proof vs. Formal Proof

Formalization becomes a tool to expose what the paper language is silently using.

PAPER PHRASE	LEAN NEEDS
"a reference frame"	Coordinate map from events to time + space; inertiality is extra structure
"the plates move with velocity v"	Extended source representation in a frame, not a particle worldline
"the same plates in their rest frame"	Co-moving frame for an extended source system
"put a test charge"	Separate localized probe with $q \neq 0$; it reads the field but is not part of the source data
"a physical speed v"	Speed-domain data: $v \in P_R$, with downward closure $u \leq v \Rightarrow u \in P_R$
"compare fields in two frames"	Field-response determination + local zero preservation

> The next slides unpack three separations: frame vs. inertial frame, source vs. probe, and field values vs. field-response comparison.

Frames Are Not Just Coordinates

A frame starts as coordinates on events; a physical system appears through its frame representation.

Minimal starting point

```
/--
An assignment which assigns each physical event to a
spacetime coordinate; distinct events cannot be
assigned the same coordinate.
-/
structure ReferenceFrame (Event : Type _) where
  coord : Event → Coord4
  sep   : Function.Injective coord
```

> No inertial mechanics is built into this structure.

Structure added later

1. **InertialFrame**
   force-free motion ⇔ uniform straight-line motion
2. **Representation in a Frame**
   we intentionally distinguish the frame from how a physical system is represented in that frame
3. **CoMovingFrame**
   an inertial frame where the representation is at rest or static

> The charged-plate proof uses both layers: inertial frames control force-free motion, while frame representations control how probes and sources are compared.

Physical Systems: Source/Probe Separation

DEFINITION **Physical system.** A physical system is a frame-independent object or source that can be represented in reference frames. We separate two frame representations with different roles:

Worldline Representation

- Used for a **localized system**, such as a test particle or small probe.
- Represented in a frame by a worldline or motion; with charge, it reads the field by Lorentz force.

↓

$F = q(E + w \times B/c)$

← FIELD FLOW

Source Representation

- Used for an **extended charge-current source**, such as the plates.
- Represented in a frame by admissible source data (ρ, j).

↓

$\text{Field } (E, B)$

This separation matters because otherwise the object used to test the field would also change the source that produces the field.

Maxwell solves the field generated by the plate source representation. The Lorentz force is then evaluated on a separate charged worldline representation.

Symmetry Fixes the Field Shape

We can use symmetry to kill most field components: **if the source is invariant under a symmetry, then the generated field must transform compatibly.**

What kills what?

translations along plates
kill dependence on x and z.

z-reflection
kills E_z, B_x, and B_y.

$R_y(\pi)T$
kills E_x.

Component parity table

	B_x	B_y	B_z	E_x	E_y	E_z
σ_{xy}	−	−	+	+	+	−
$R_y(\pi)T$	+	−	+	−	+	−
survives	✗	✗	✓	✗	✓	✗

The listed symmetries preserve the moving-plate source. A − therefore forces that component to vanish. Here E is polar, while B is axial.

> Symmetry is a natural interface which turns source geometry into reusable field-shape lemmas: $E = E_y\,\hat{y}$, $B = B_z\,\hat{z}$. Maxwell laws later fix the coefficients.

When Formalization Catches a Hidden Assumption

Paper shortcut by Wu: in the plates' co-moving frame: $B' = 0$ by time reversal, and $E' \neq 0$.

ACCEPTED BY SYMMETRY

$B' = 0$

In the plates' co-moving frame, the source is static. Since magnetic field is odd under time reversal, it vanishes.

NOT AUTOMATIC

Why is $E' \neq 0$?

We cannot simply assume Maxwell equations again in this frame. The explicit Maxwell was only assumed in the lab frame R.

assume $(E', B') = (0,0)$ —linearity: zero maps to zero⇒→ **contradicts nonzero response in R**

Formalization forces every hidden physical step to say which assumption justifies it. This leads to axiom audit.

What Makes a Physical Axiom Good?

In the spirit of **Hilbert's sixth problem**, when Lean exposes a missing step in which we need a new assumption, we ask is it a general principle or just a patch?

LOCAL PATCH

assume the missing step directly

- ✕ high target overlap
- ✕ no explanation
- ✕ hard to reuse

EXAMPLES

`E = Ey ŷ, B = Bz ẑ`

`E′ ≠ 0`

`proof gap`

Lean

`axiom audit`

Lean turns a gap into an audit question

STRUCTURAL GENERATOR

find the principle that generates the missing step

- ✔ low target overlap
- ✔ explanatory
- ✔ generates reusable lemmas

`structural principle`

↓

`local seam 1` `local seam 2` `local seam 3`

EXAMPLES

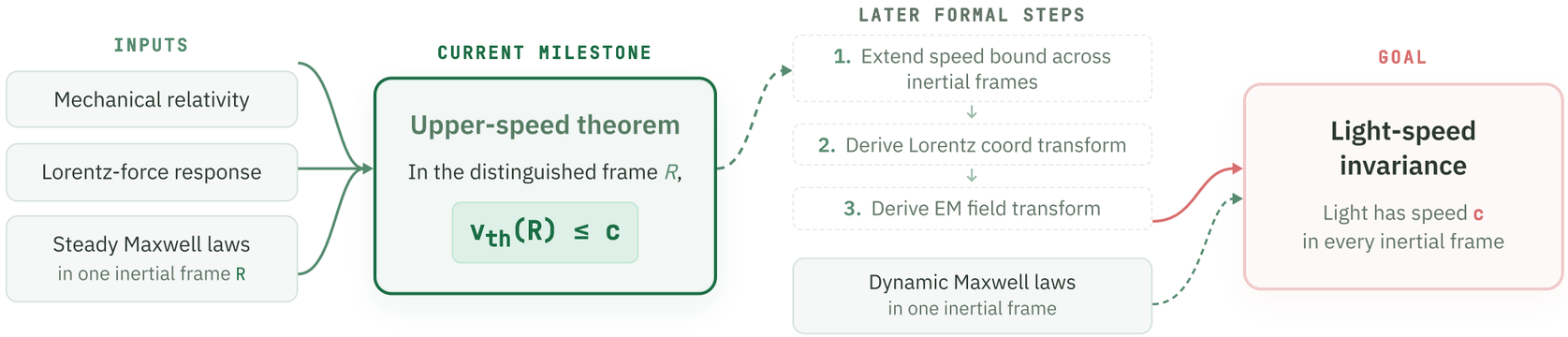
`source symmetry + parity`

`field-response comparison`

> Formalization not only asks what assumption is missing, it also helps judge the quality of the assumption. A good axiom often becomes part of the interface.

Where the Upper-Speed Theorem Fits

The charged-plate argument proves a speed bound first. The larger route continues toward light-speed invariance.



> **Formalization turns a paper derivation into an auditable route: which assumptions and interfaces are given and what still remains.**

Closing: From Verification to Interface Discovery

Lean is not only useful for checking proofs, but also making both interfaces and assumptions explicit.

Informal Reasoning
roles and assumptions are compressed in natural language

→

Proof Checking
Lean verifies formal statements once the statement is fixed

→

Axiom Audit
when a hidden step appears, we ask what assumption justifies it.

⇄ mutual

Interface Discovery
when the right assumption is found, it may give rise to reusable interfaces.

Thank you!

LeanCat BENCHMARK
arXiv: 2512.24796 · GitHub

Relativity ONGOING
Special Relativity in Lean 4